

Lec2

如何对计算过程抽象，我们从**finite function**开始。

✓ "Definition"

finite function:

$$f : \{0, 1\}^n \rightarrow \{0, 1\}^m$$

即输入和输出长度均固定。

Model1: Boolean Circuit

boolean circuit由三种门构成，对应三个最基本的函数：

$$\text{AND}(a, b) = \begin{cases} 1, a = b = 1 \\ 0, \text{otherwise} \end{cases}$$

$$\text{OR}(a, b) = \begin{cases} 0, a = b = 0 \\ 1, \text{otherwise} \end{cases}$$

$$\text{NOT}(a) = \begin{cases} 1, a = 0 \\ 0, a = 1 \end{cases}$$

通过这三个门可以组成更多复杂函数。

☰ Example

XOR:

异或

$$\text{XOR}(a, b) = \overline{(a \wedge b)} \wedge (a \vee b)$$

MAJ3:

看三个输入中的1多还是0多，返回多的那个。

$$\text{MAJ3}(x_0, x_1, x_2) = \begin{cases} 1, x_1 + x_2 + x_3 \geq 2 \\ 0, \text{otherwise} \end{cases} = (x_0 \wedge x_1) \vee (x_1 \wedge x_2) \vee (x_2 \wedge x_0)$$

我们可以将boolean circuit抽象成一个有向无环图，一共有三种节点：

- 输入节点：入度为0，出度 ≥ 1 ，有 n 个： $X[0], X[1], \dots, X[n-1]$
- 门节点：与门和或门入度为2，非门入度为1，出度均 ≥ 1 ，有 s 个
- 输出节点：入度为1，出度为0，有 m 个： $Y[0], Y[1], \dots, Y[m-1]$

✎ Note

输出节点实际是在门节点中指定。

电路的规模规定为门的个数，即 $|C| = s$ 。

当我们拿到长度为 n 的01串 $x \in \{0, 1\}^n$ ，我们将每一位都输入到对应的输入节点中： $x_0 \rightarrow X[0], x_1 \rightarrow X[1], \dots, x_{n-1} \rightarrow X[n-1]$ ，然后输出节点就会有输出值： $Y[0], Y[1], \dots, Y[m-1]$ ，我们将这些输出值拼接起来得到长度为 m 的01串，即为电路 C 在输入 x 下的输出，记为 $C(x)$ 。

我们说电路 C 计算了某个函数 $f: \{0, 1\}^n \rightarrow \{0, 1\}^m$ ，当且仅当对于任意 $x \in \{0, 1\}^n$ ，都有 $C(x) = f(x)$ 。

以XOR为例，我们可以将电路表示为如下程序：

```
temp1 = AND(X[0], X[1])
temp2 = NOT(temp1)
temp3 = OR(X[0], X[1])
Y[0] = AND(temp2, temp3)
```

这一段程序称为**AON-CIRC program**，和原本的电路等价，可以相互转换。

我们可以观察到，程序的行数就是门的个数，因此程序的规模定义为： $|P| = \#lines = s$

Theorem1

一个函数 f 可以用一个有 s 个门的boolean circuit计算，当且仅当它可以用一个有 s 行的AON-CIRC program计算。

Model2: NAND Circuit

还有一种很重要的门——与非门：

NAND(a,b) = { 0, a = b = 1; 1, otherwise

NAND门可以用AND和NOT门实现；反过来，AND门、OR门、NOT门也都可以用NAND门实现：

NOT(a) = NOT(AND(a,a)) = NAND(a,a)
AND(a,b) = NOT(NAND(a,b)) = NAND(NAND(a,b),NAND(a,b))
OR(a,b) = NOT(AND(NOT(a),NOT(b))) = NAND(NAND(a,a),NAND(b,b))

因此，同样可以构成NAND circuit，和boolean circuit是等价的。

如果NAND circuit有 s 个门，则将其转化成boolean circuit后，最多有 $2s$ 个门。（一个NAND门转化成一个AND门和一个NOT门）

如果boolean circuit有 s 个门，则将其转化成NAND circuit后，最多有 $3s$ 个门。（三种门中OR门开销最大，转化成3个NAND门）

类似的，对于NAND circuit，我们也可以定义对应的等价程序**NAND-CIRC program**。我们仍以XOR函数为例，其在NAND circuit下的电路图如下：

对应的NAND-CIRC program为：

```
temp1 = NAND(X[0], X[1])
temp2 = NAND(X[0], temp1)
temp3 = NAND(X[1], temp1)
Y[0] = NAND(temp2, temp3)
```

规模与复杂度

Universal

如果考虑电路和程序，我们有四种计算模型：

- boolean circuit
- AON-CIRC program
- NAND circuit
- NAND-CIRC program

这些模型是universal的，即它们可以计算任意finite function。

给出函数集合 $\mathcal{F} = \{f_1, f_2, \dots, f_k\}$ ，若要判断 $\mathcal{F}$ 是否universal，我们只需要判断NAND能否变成这些函数。

☰

Example

$$\begin{cases} f_1 : \text{ONE}(a, b) = 1 \\ f_2 : \text{ZERO}(a, b) = 0 \\ f_3 : \text{IF}(a, b, c) = \begin{cases} b, a = 1 \\ c, a = 0 \end{cases} \end{cases}$$

这三个函数拼成的集合 $\mathcal{F}$ 是universal的。

原因： $\text{NAND}(a, b) = \text{IF}(a, \text{IF}(b, \text{ZERO}(a, b), \text{ONE}(a, b)), \text{ONE}(a, b))$

也就是说，任意一个有限函数都可以转换成函数集合表示。

Theorem2

对于任意 $n, m > 0$ 和任意有限函数 $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ ，存在一个boolean circuit C 可以计算 f 。

那么 C 最多需要多少个门才能计算这个函数？

$O(m \cdot n \cdot 2^n)$

🔥

"proof"

考虑输出 $Y[j]$ 和输入 $X[0], X[1], \dots, X[n - 1]$ 的对应关系。

$X[0]$	$X[1]$	...	$X[n - 2]$	$X[n - 1]$	$Y[j]$
0	0	...	0	0	0
0	0	...	0	1	0
0	0	...	1	0	1
...	...	...	...	...	...

上述表格只是举个例子，我们发现， $Y[j]$ 可以表示成析取范式：

$$(\dots \wedge \dots \wedge \dots) \vee (\dots \wedge \dots \wedge \dots) \vee \dots$$

每一个括号对应 $X[0], X[1], \dots, X[n - 1]$ 的一个特定取值组合，因此总共有 2^n 个括号。

每一个括号中有 n 个变量，因此在这个析取范式中与门和或门的数量级为 $O(n \cdot 2^n)$ 。

我们只考虑了一个输出 $Y[j]$ ，总共有 m 个输出，因此总的门数为 $O(m \cdot n \cdot 2^n)$ 。

那么， $O(m \cdot n \cdot 2^n)$ 这个界能否改进，对于特定的函数是否又有不同的界？

我们用NAND-CIRC program来思考这个问题。我们引入一些语法糖，这些语法并不会实质性地改变模型的内容。

语法糖1：loop of fixed length 固定次数的循环

```
for i in range(n):  
    ...  
    ...  
    ...  
    ...
```

假设循环体内有4行，则实际对应原本的 $4n$ 行。

语法糖2：user-defined procedure 自定义过程

```
def MAJ3(a,b,c):
    ...
    ...
    ...
```

语法糖3：conditional statement 条件判断

```
if (cond):
    a = ... (1)
else:
    a = ... (2)
```

如果(1)处对应原本的 l_1 行，(2)处对应原本的 l_2 行，则实际对应原本的 $l_1 + l_2 + c$ 行，其中 c 为常数。

 Example

ADD : $\{0,1\}^{2n} \rightarrow \{0,1\}^{n+1}$
 $ADD(x_0, \dots, x_{2n-1}) = x_0 \dots x_{n-1} + x_n \dots x_{2n-1}$

从program角度理解：

```
Result = [0] * (n+1)
Carry = [0] * (n+1)
for i in range(n):
    Result[i] = XOR(Carry[i], XOR(X[i], X[n+i]))
    Carry[i+1] = MAJ3(Carry[i], X[i], X[n+i])
Result[n] = Carry[n]
return Result
```

现在考虑这段程序如果展开需要多少行（即多少个门）。

循环外的其他部分和循环内部是常数行，则整个循环体展开成NAND-CIRC program是 $O(n)$ 行，因此实现ADD需要 $O(n)$ 个门。

 Example

MULT : $\{0,1\}^{2n} \rightarrow \{0,1\}^{2n}$
 $MULT(x_0, \dots, x_{2n-1}) = x_0 \dots x_{n-1} \times x_n \dots x_{2n-1}$

我们可以将乘法看作最多 n 次加法（竖式），因此实现MULT需要 $O(n^2)$ 个门。

 Note

但实际上有机会降到 $O(n^{\log_2 3})$ 甚至更低。

LOOKUP函数

现在我们来看一个很重要的函数——LOOKUP : $\{0,1\}^{2^k+k} \rightarrow \{0,1\}$

输入分为两个部分。第一部分长度为 2^k ，记作 X ；第二部分长度为 k ，记作 i 。

LOOKUP的功能是：返回 X 中第 i 位的值。

采用的归纳的思想：

对于LOOKUP₁($X[0], X[1], i[0]$)

若 $i[0] = 0$ ，则返回 $X[0]$ ；

若 $i[0] = 1$ ，则返回 $X[1]$ 。

规模为 $O(1)$ 。

对于 $\text{LOOKUP}_2(X[0], X[1], X[2], X[3], i[0], i[1])$

若 $i[0] = 0$ ，则返回 $\text{LOOKUP}_1(X[0], X[1], i[1])$ ；

若 $i[0] = 1$ ，则返回 $\text{LOOKUP}_1(X[2], X[3], i[1])$ 。

对于 $\text{LOOKUP}_k(X[0], X[1], \dots, X[2^k - 1], i[0], i[1], \dots, i[k - 1])$

若 $i[0] = 0$ ，则返回 $\text{LOOKUP}_{k-1}(X[0], X[1], \dots, X[2^{k-1} - 1], i[1], \dots, i[k - 1])$ ；

若 $i[0] = 1$ ，则返回 $\text{LOOKUP}_{k-1}(X[2^{k-1}], X[2^{k-1} + 1], \dots, X[2^k - 1], i[1], \dots, i[k - 1])$ 。

通过归纳可得： LOOKUP_k 的规模为 $O(2^k)$ 。